

≡ REACT.JS ≡

COMPLETE HANDBOOK

"Learn once,
write anywhere,
use everywhere."

1. INTRODUCTION TO REACT.JS

PAGE 1

What is React?

React is a JavaScript library for building user interfaces, especially single-page applications (SPAs).

It is maintained by Facebook (Meta) and used by thousands of companies worldwide.

Key Facts

- Library (not a framework)
- Declarative and Component-based
- Fast, Scalable and Flexible
- Uses Virtual DOM for performance
- Large ecosystem and community

Why React?

- ★ Reusable components
- ★ Virtual DOM for better performance
- ★ Unidirectional data flow
- ★ Easy to learn and integrate
- ★ Huge ecosystem (React Router, Redux, etc.)
- ★ Strong developer tools (React DevTools)
- ★ Backed by Meta with long-term support



Traditional vs React

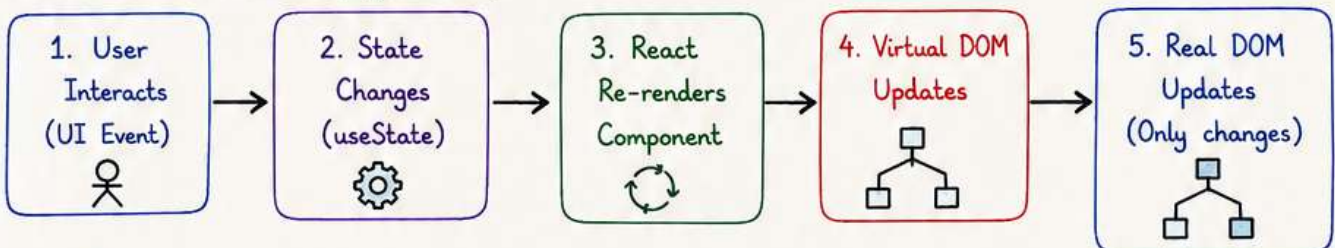
Traditional (Multi Page)

- Every action → new page request
- Full page reload
- Slower UX

React (SPA)

- Updates only what changes
- No full page reload
- Faster & smoother UX

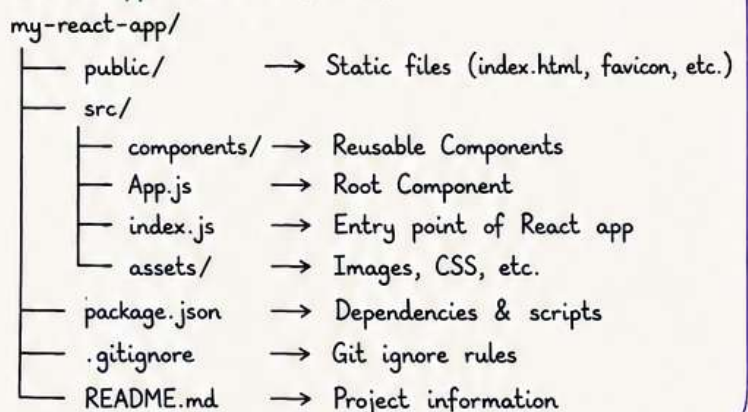
How React Works? (High Level)



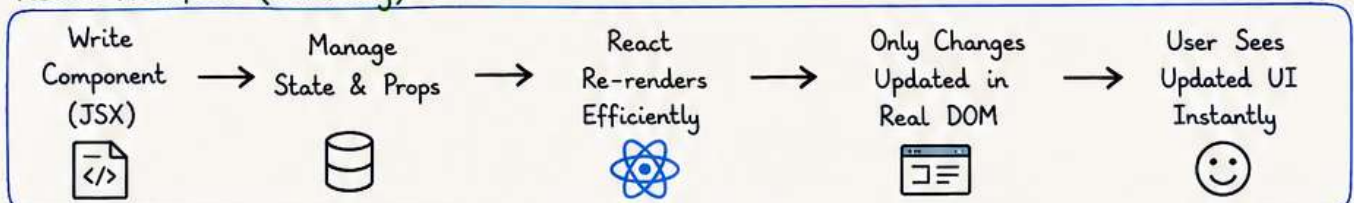
Key Terms

- **Component**: Building block of UI.
- **JSX**: JavaScript XML - syntax extension.
- **Props**: Read-only inputs passed to components.
- **State**: Data managed within a component.
- **Virtual DOM**: Lightweight copy of real DOM.
- **Reconciliation**: Process of finding changes between Virtual DOM and Real DOM.
- **Unidirectional Data Flow**: Data flows in one direction (Parent → Child).

React App Structure (Basic)



React Workflow (Summary)



React is all about building reusable components and managing UI efficiently.

IG: abhi_techhub



2. REACT FUNDAMENTALS

PAGE 2

Before going deep, let's understand the basic building blocks of React.

2.1 JSX (JavaScript XML)

JSX allows us to write HTML-like code in JavaScript. It makes UI structure easy and readable.

```
const element = (  
  <div className="card">  
    <h1>Hello React!</h1>  
    <p>JSX makes it simple.</p>  
  </div>  
);
```

JSX gets
transpiled to
React.createElement()
behind the scenes.

```
ReactDOM.render(element, document.getElementById('root'));
```

2.2 Components

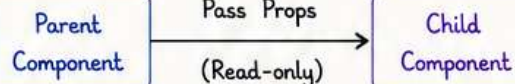
Components are reusable pieces of UI. Every React app is built using components.

```
function Welcome(props) {  
  return (  
    <div className="welcome">  
      <h2>Hello, {props.name}!</h2>  
    </div>  
  );  
}
```

★ Components can accept inputs (props) and return JSX to describe the UI.

2.3 Props (Properties)

Props are read-only inputs passed from parent to child component.



Example:

```
// Parent  
<UserCard name="Abhi" age={24} />  
  
// Child  
function UserCard(props) {  
  return <p>{props.name} is {props.age} years old.</p>;  
}
```

2.4 State

State is mutable data managed within a component. When state changes, the component re-renders.

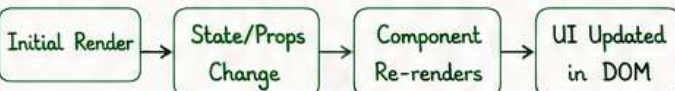
```
import { useState } from 'react';  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <h2>Count: {count}</h2>  
      <button onClick={() => setCount(count + 1)}>  
        Increment  
      </button>  
    </div>  
  );  
}
```

State variable

Update state
using setter
function

2.5 Rendering

React renders UI based on state and props.



★ Note:

- React updates only the parts of UI that actually changed (Virtual DOM).
- This makes React fast and efficient.



2.6 KEY TAKEAWAYS

- ✓ React is a library, not a framework.
- ✓ Build UI using components.
- ✓ JSX makes code readable.
- ✓ Props are read-only inputs.
- ✓ State is mutable and triggers re-render.
- ✓ React is declarative and efficient.



Tip: Think in components. Break UI into small, reusable pieces. That's the React way!

IG: abhi_techhub



3. COMPONENTS DEEP DIVE

PAGE 3

Components are the core building blocks of React apps.

3.1 Types of Components

There are two main types of components.

Functional Component

- Simple JavaScript function
- Uses Hooks
- No lifecycle methods
- Easier to write and test

(Preferred)

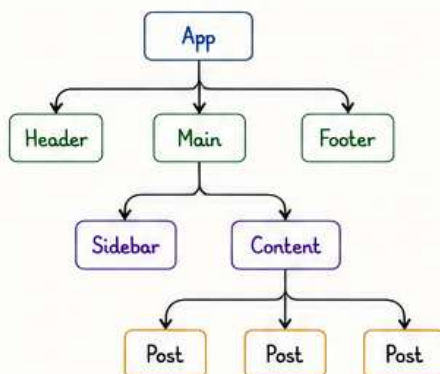
Class Component

- ES6 class that extends React.Component
- Has lifecycle methods
- More boilerplate
- Used in older codebases

(Less Preferred)

3.2 Component Composition

Build complex UI by combining small components.



★ Break UI into small, reusable components.

3.3 Reusability

Components can be reused anywhere in the app.

```

function Button(props) {
  return (
    <button className={props.type}>
      {props.label}
    </button>
  );
}

// Reuse
<Button type="primary" label="Login" />
<Button type="secondary" label="Cancel" />
  
```

Same component, different props.

★ Reusable components = Less code, easier maintenance, consistent UI.

3.4 Component Lifecycle (Class Component)

Class components have lifecycle methods.



★ Functional components don't have lifecycle methods. Use useEffect() Hook instead.

3.5 Functional vs Class Component

Feature	Functional Component	Class Component
Syntax	JavaScript function	ES6 class
State	useState Hook	this.state
Lifecycle	useEffect Hook	Lifecycle methods
Performance	Better (less code)	Slightly heavy
Code Size	Less boilerplate	More boilerplate
Preferred	Yes (Modern React)	No (Legacy)

3.6 When to Use What?

Use Functional Component when:

- ✓ New projects
- ✓ Simple UI
- ✓ Need Hooks
- ✓ Better performance
- ✓ Easier testing

Use Class Component when:

- ✓ Working on legacy code
- ✓ Need error boundaries
- ✓ Complex lifecycle (rare cases)

3.7 Best Practices

- ✓ Keep components small and focused.
- ✓ One component = one responsibility.
- ✓ Use meaningful names.
- ✓ Reuse components using props.
- ✓ Keep UI logic inside the component.

Example Structure

```

components/
├── Header.jsx
├── Footer.jsx
├── Button.jsx
├── Card.jsx
└── Modal.jsx
  
```

3.8 Key Takeaways

- ★ Components are reusable building blocks of React.
- ★ Prefer functional components with Hooks.
- ★ Compose small components to build big UI.
- ★ Reusability = Less code + Easy maintenance.
- ★ Follow best practices for clean and scalable code.



Remember: Think in components, not in pages.
Break UI into small parts and compose them.

IG: abhi_techhub



4. REACT HOOKS - PART 1

PAGE 4

Hooks let you use state and other React features in functional components.

4.1 useState() - Manage State

- Adds state to functional components.
- Returns a pair: [state, setState]

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0); // Initial state
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

- ★ State updates are asynchronous.
React may batch multiple updates for performance.

4.3 useRef() - Access DOM & Persist Values

- Returns a mutable ref object.
- Does not cause re-render when updated.

```
import { useRef } from 'react';
function InputFocus() {
  const inputRef = useRef(null); // initial value
  const focusInput = () => {
    inputRef.current.focus(); // access DOM
  };
  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={focusInput}> Focus Input</button>
    </div>
  );
}
```

useRef returns { current: value }

- ★ useRef is also used to store values that don't need to trigger re-render.

EXAMPLE - COUNTER WITH EFFECT & REF

```
function CounterExample() {
  const [count, setCount] = useState(0);
  const renderCount = useRef(0); // Tracks how many times component rendered without causing re-render
  useEffect(() => {
    renderCount.current = renderCount.current + 1;
    console.log("Rendered:", renderCount.current);
  });
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}> Increment</button>
    </div>
  );
}
```

4.2 useEffect() - Side Effects

- Runs side effects in functional components.
- Used for API calls, subscriptions, DOM updates, etc.

```
// Run after every render
useEffect(() => {
  console.log("Component rendered");
});

// Run only once (on mount)
useEffect(() => {
  fetchData();
}, []);

// Run when 'count' changes
useEffect(() => {
  console.log("Count changed:", count);
}, [count]);
```

No dependency array

Empty array = once

Runs when count changes

Effect cleanup (on unmount or before next effect):

```
useEffect(() => {
  const timer = setInterval(() => console.log("Tick"), 1000);
  return () => clearInterval(timer); // cleanup
}, []);
```

HOOKS QUICK SUMMARY

Hook	Purpose	Returns	Triggers Re-render?
useState	Manage state	[state, setState]	Yes (on setState)
useEffect	Side effects	Nothing	Yes (depending on dependency)
useRef	DOM access / persist value	{ current }	No

BEST PRACTICES

- ✓ Use useState for state management.
- ✓ Use useEffect for side effects and cleanup.
- ✓ Always provide dependency array to useEffect.
- ✓ Use useRef for DOM access or storing mutable values.
- ✓ Keep components small and focused.
- ✓ Don't overuse useEffect (not for every state change).



COMMON MISTAKES

- ✗ Forgetting dependency array in useEffect.
→ Causes unnecessary renders or infinite loops.
- ✗ Updating state inside useEffect without proper condition.
→ May cause infinite re-render.
- ✗ Using useState when useRef is enough.
→ Leads to extra re-renders.



Tip: Hooks let you write cleaner, shorter and more powerful React code.

IG: abhi_techhub



5. REACT HOOKS - PART 2

PAGE 5

More powerful Hooks to build scalable and maintainable React applications.

5.1 useMemo() - Memoize Expensive Values

- Memoizes a computed value.
- Re-computes only when dependencies change.

```
import { useMemo } from 'react';
function ExpensiveCalc({ list }) {
  const result = useMemo(() => {
    console.log("Calculating...");
    return list.reduce((sum, n) => sum + n, 0);
  }, [list]); // Recompute only when 'list' changes

  return <p>Sum: {result}</p>;
}
```

Saves
expensive
re-calculation

★ Great for heavy computations.

5.2 useCallback() - Memoize Functions

- Returns a memoized function.
- Prevents child re-renders when function is passed as prop.

```
import { useState, useCallback } from 'react';
function Parent() {
  const [count, setCount] = useState(0);

  const increment = useCallback(() => {
    setCount(c => c + 1);
  }, []); // Same function reference

  return <Child onIncrement={increment} />;
}
```

Same
function
reference
until deps
change

★ Useful with React.memo to avoid unnecessary re-renders.

5.3 useContext() - Share Data Globally

- Lets you access context value without prop drilling.
- Works with Context API.

```
import { createContext, useContext } from 'react';
const ThemeContext = createContext();

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <p>Theme: {theme}</p>;
}
```

Access
context
value
directly

★ Best for global data like theme, user, language, etc.

5.4 Custom Hooks - Reuse Logic

- Let you extract and reuse component logic.
- Start with "use" keyword.

```
// useWindowSize.js
import { useState, useEffect } from 'react';

export default function useWindowSize() {
  const [size, setSize] = useState(window.innerWidth);

  useEffect(() => {
    function handleResize() {
      setSize(window.innerWidth);
    }
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return size;
}
```

Reusable
logic as
a Hook

★ Custom Hooks = Clean, reusable and testable logic.

COMMONLY USED HOOKS OVERVIEW

Hook	Purpose	Returns	Triggers Re-render?
useState	Manage state	[state, setState]	Yes (on setState)
useEffect	Side effects	Nothing	Yes (on deps change)
useRef	Access DOM / store values	{ current }	No
useMemo	Memoize value	Memoized value	No (until deps change)
useCallback	Memoize function	Memoized function	No (until deps change)
useContext	Consume context	Context value	Yes (on context change)
Custom Hook	Reuse logic	Anything	Depends on logic

BEST PRACTICES

- ✓ Keep dependency arrays accurate.
- ✓ Use Hooks at the top level (not inside loops/conditions).
- ✓ Create custom Hooks to reuse logic.
- ✓ Combine Hooks for clean and powerful components.
- ✓ Prefer functional components with Hooks.



WHEN TO USE WHAT?

- useState → Local state in component.
- useEffect → API calls, subscriptions, DOM updates.
- useRef → Access DOM, store mutable values without re-render.
- useMemo → Expensive calculations.
- useCallback → Functions passed to child components.
- useContext → Global data (theme, user, settings).
- Custom Hooks → When logic is reused in multiple places.

COMMON MISTAKES

- ✗ Missing dependencies in useEffect / useMemo / useCallback.
- ✗ Overusing useEffect for state changes.
- ✗ Using Hooks inside loops, conditions or nested functions.
- ✗ Creating functions inside JSX without useCallback.



Tip: Hooks make React code cleaner, reusable and more powerful. Use the right Hook for the right job.

IG: abhi_techhub



6. STATE MANAGEMENT

PAGE 6

Choosing the right state management is key to building scalable React applications.

6.1 Types of State

1. Local State

Managed inside a component using `useState`.

2. Global State

Shared across multiple components.

Example:

Local State → Input value, modal open/close, counter.



Global State → User info, theme, cart, auth status.



6.2 When to Use What?

Solution	Use When	Pros	Cons
<code>useState</code>	Component level state only	Simple, built-in, easy to use	Not shareable
Context API	Share state across many components	No extra libs, built-in	Re-renders can affect performance
Redux (Toolkit)	Complex app, many features, frequent updates	Predictable, DevTools, middleware, scalable	More boilerplate (less with Redux Toolkit)
Zustand	Simple global state with less boilerplate	Lightweight, easy, fast performance	Smaller ecosystem compared to Redux

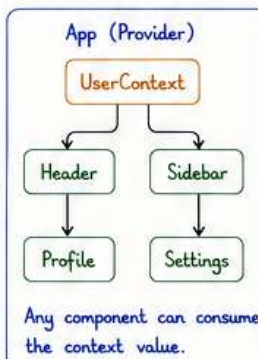
6.3 Local State with `useState`

```
import { useState } from 'react';

function Toggle() {
  const [isOn, setIsOn] = useState(false);
  return (
    <div>
      <p>Status: {isOn ? 'ON' : 'OFF'}</p>
      <button onClick={() => setIsOn(!isOn)}>
        Toggle
      </button>
    </div>
  );
}
```

State is local to this component only.

6.4 Global State with Context API

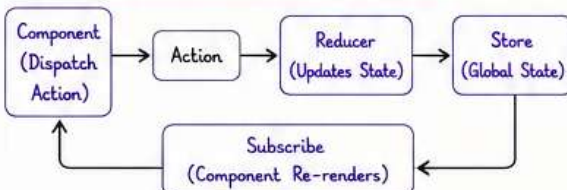


```
// Create Context
const UserContext = createContext();

// Provider
<UserContext.Provider value={{user, setUser}}>
  <App />
</UserContext.Provider>

// Consume in any component
const { user, setUser } = useContext(UserContext);
```

6.5 Redux (Toolkit) - Overview



Example Flow:

1. User clicks "Add to Cart"
2. Action is dispatched /
3. Reducer updates the store
4. Store notifies subscribed components
5. Components re-render with new data



6.6 State Management Comparison

Feature	<code>useState</code>	Context API	Redux Toolkit	Zustand
Scope	Local	Global	Global	Global
Boilerplate	Very Low	Low	Medium	Very Low
Performance	High	Medium	High	High
DevTools	No	No	Yes	Yes (Plugins)
Best For	Simple UIs	Medium Apps	Large Apps	Simple to Medium Apps

BEST PRACTICES

- ✓ Keep state as local as possible.
- ✓ Lift state up only when needed.
- ✓ Use Context for medium scale apps.
- ✓ Use Redux (Toolkit) for complex & large scale apps.
- ✓ Avoid prop drilling - prefer Context.
- ✓ Keep state structure normalized.
- ✓ Don't store derived data - compute it.



COMMON MISTAKES

- ✗ Overusing global state for simple UI.
- ✗ Storing too much data in Context.
- ✗ Not normalizing data in Redux store.
- ✗ Mutating state directly.
- ✗ Using multiple state management libraries in one app.



Remember: Use the simplest solution that works. You can always migrate to a stronger solution when your app grows.

IG: abhi_techhub

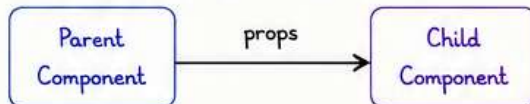


7. COMPONENT COMMUNICATION

React components communicate with each other in different ways.

7.1 Parent → Child (Props)

- Parent passes data to child using props.
- Child receives props as input.



```

// Parent.js
function Parent() {
  const user = { name: "Abhi", age: 24 };
  return <Child user={user} />;
}

// Child.js
function Child({ user }) {
  return <div>{user.name} - {user.age}</div>;
}
  
```

7.2 Child → Parent (Callback Props)

- Child sends data back to parent using callback function.
- Parent passes the function as props.



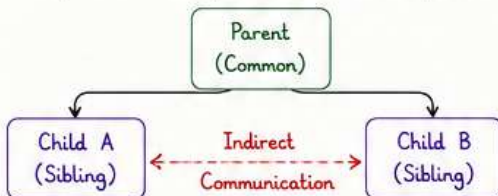
```

// Parent.js
function Parent() {
  const handleChildData = (data) => {
    console.log("Data from child:", data);
  };
  return <Child onSendData={handleChildData} />;
}

// Child.js
function Child({ onSendData }) {
  const send = () => onSendData("Hello Parent");
  return <button onClick={send}>Send</button>;
}
  
```

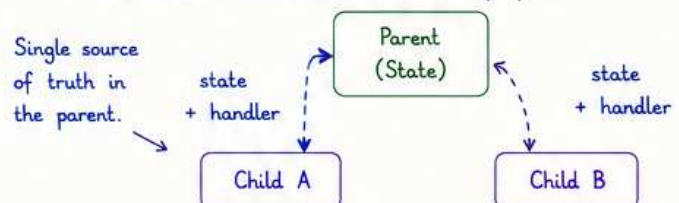
7.3 Sibling Communication

- Siblings don't have direct access to each other.
- Lift the state up to their common parent.



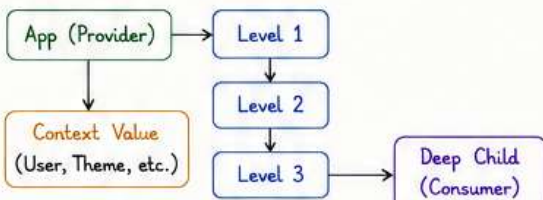
7.4 Lifting State Up

- Move shared state to the closest common parent.
- Pass state and handlers down as props.



7.5 Context API for Deep Hierarchy

- Avoid prop drilling in deeply nested components.
- Provide global data using Context API.



Any level can access the context value.

BEST PRACTICES

- ✓ Keep state as local as possible.
- ✓ Lift state up only when needed.
- ✓ Use Context API for global or cross-cutting data.
- ✓ Avoid unnecessary re-renders.
- ✓ Keep components small and focused.



7.6 Choosing the Right Approach

Scenario	Best Approach	Why?
Parent → Child	Props	Simple and straightforward
Child → Parent	Callback Props	Allows child to send data up
Sibling → Sibling	Lift State Up	Parent acts as bridge
Deeply Nested Components	Context API	Avoid prop drilling, clean and scalable

COMMON MISTAKES

- ✗ Passing too many props (prop drilling).
- ✗ Storing global data in props.
- ✗ Not lifting state up for shared data.
- ✗ Overusing Context API for small state.



Tip: Good communication between components makes your code clean, scalable and easy to maintain.



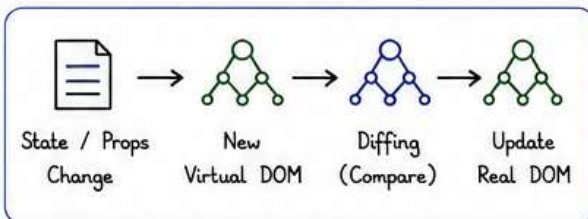
8. REACT RENDERING & PERFORMANCE

PAGE 8

Understanding how React renders and how to optimize it is key to building fast apps.

8.1 How React Renders?

- When state or props change, React re-renders the component.
- It builds a new Virtual DOM.
- Compares it with previous Virtual DOM (Diffing).
- Updates only what changed (Reconciliation).
- Updates the Real DOM.



8.2 Key Concepts

- **Virtual DOM:** Lightweight JS representation of real DOM.
- **Diffing:** Finds differences between old and new Virtual DOM.
- **Reconciliation:** Efficiently updates only the changed parts.
- **Fiber Architecture:** React 16+ new engine for better scheduling and performance.

8.3 Re-rendering Triggers

- State update using `setState` or `useState`.
- Props change from parent.
- Context value change.
- Parent component re-render.
- Force update (not recommended).

★ If parent re-renders, child re-renders too (by default). This can hurt performance in large apps.

8.4 Keys in Lists

- Keys help React identify which items changed, added or removed.
- Always use stable and unique keys.

Good 👍

```
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

Using index as key can cause unnecessary re-renders and bugs.

Bad 👎

```
{items.map((item, index) => (
  <li key={index}>{item.name}</li>
))}
```

8.6 `useMemo()` - Memoize Expensive Value

- Memoizes the result of expensive calculations.
- Re-computes only when dependencies change.

```
const expensiveValue = useMemo(() => {
  // heavy computation
  return doSomethingExpensive(data);
}, [data]);
```

★ Use `useMemo` for values, not for every simple calculation.

8.5 `React.memo()`

- Prevents re-rendering of functional components if props didn't change.
- Uses shallow comparison.

```
const Child = React.memo(function ChildComponent({ name }) {
  console.log("Child rendered");
  return <div>{name}</div>;
});
```

★ Child re-renders only when props change.

8.7 `useCallback()` - Memoize Function

- Returns a memoized callback function.
- Prevents child re-renders when function is passed as prop.

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, [dependency]);
```

★ Use `useCallback` when passing functions to optimized children (like memo components).

8.8 `React.memo` vs `useMemo` vs `useCallback`

Feature	<code>React.memo</code>	<code>useMemo</code>	<code>useCallback</code>
Type	Higher Order Component	Hook	Hook
Use For	Prevent component re-render	Memoize expensive value	Memoize function
Returns	Memoized Component	Memoized Value	Memoized Function
Re-renders when	Props change (shallow compare)	Dependencies change	Dependencies change
Best For	Pure components	Heavy computations	Functions passed as props

8.9 Performance Best Practices

- ✓ Use `React.memo` for pure components.
- ✓ Use `useMemo` for expensive calculations.
- ✓ Use `useCallback` for functions passed as props.
- ✓ Avoid creating objects/arrays inline in JSX.
- ✓ Use keys properly in lists.
- ✓ Keep component state as local as possible.
- ✓ Split large components into smaller ones.
- ✓ Avoid unnecessary re-renders.



COMMON MISTAKES

- ✗ Using index as key in lists.
- ✗ Using `useMemo`/`useCallback` without need.
- ✗ Over-optimizing small and simple components.
- ✗ Ignoring dependency array in Hooks.



Remember: Optimize where it matters. Measure → Identify → Optimize.
Don't guess. Use React DevTools Profiler.

IG: abhi_techhub



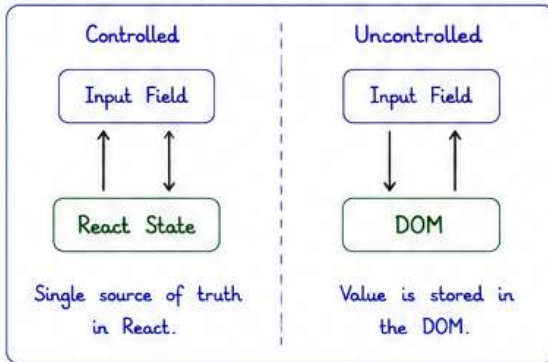
9. FORMS & EVENTS

PAGE 9

Forms and events are the backbone of user interaction in React.

9.1 Controlled vs Uncontrolled Components

- Controlled: Form data is handled by React state.
- Uncontrolled: Form data is handled by the DOM itself.



9.3 Uncontrolled Component Example

- React does not control the input value.

```
import { useRef } from 'react';

function UncontrolledForm() {
  const inputRef = useRef();

  const handleSubmit = (e) => {
    e.preventDefault();
    alert('Hello ' + inputRef.current.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" ref={inputRef} placeholder="Enter name" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Value is read from the DOM when needed.

9.2 Controlled Component Example

- Every keystroke updates the state.

```
import { useState } from 'react';

function Form() {
  const [name, setName] = useState('');

  const handleChange = (e) => {
    setName(e.target.value);
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    alert('Hello ' + name);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" value={name} onChange={handleChange} placeholder="Enter your name" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

State holds the input value.

Update state on every change.

Submit form data.

9.4 Handling Events

- Events in React are written in camelCase.
- Pass a function, not a string.

DOM Event	React Event
onclick	onClick
onchange	onChange
onsubmit	onSubmit
oninput	onInput
onkeydown	onKeyDown
onfocus	onFocus
onblur	onBlur

Example:

```
<button onClick={handleClick}>
  Click Me
</button>
```

★ Always pass the function reference, not call it.

Wrong: `onClick={handleClick()}` ✗
Right: `onClick={handleClick}` ✓

9.5 Form Validation (Simple Example)

- Validate before submitting.
- Show error messages to the user.

```
const [email, setEmail] = useState('');
const [error, setError] = useState('');

const handleSubmit = (e) => {
  e.preventDefault();
  if (!email.includes('@')) {
    setError('Enter a valid email');
    return;
  }
  setError('');
  alert('Form submitted!');
};
```

Output UI:

Email:

Enter a valid email

★ Validations improve user experience and data quality.

9.6 Common Form Inputs in React

Input Type	How to Use in React
Text	<code><input type="text" /></code>
Textarea	<code><textarea value={text} onChange={...} /></code>
Checkbox	<code><input type="checkbox" checked={isChecked} onChange={...} /></code>
Radio	<code><input type="radio" name="gender" value="male" /></code>
Select	<code><select value={city} onChange={...}>...</select></code>
File	<code><input type="file" onChange={...} /></code>

BEST PRACTICES

- ✓ Use controlled components for most forms.
- ✓ Validate user input before submission.
- ✓ Show meaningful error messages.
- ✓ Keep form state simple and flat.
- ✓ Use useRef for file inputs and complex cases.
- ✓ Prevent default behavior in forms (`e.preventDefault()`).
- ✓ Keep event handlers clean and reusable.



COMMON MISTAKES

- ✗ Mixing controlled and uncontrolled inputs.
- ✗ Forgetting value or onChange in controlled inputs.
- ✗ Not preventing default form submission.
- ✗ Doing heavy logic directly in onChange.
- ✗ Storing sensitive data in client-side state.



Tip: Good forms = Happy users.
Validate early, handle errors gracefully.



10. EFFECT HOOK & LIFECYCLE

PAGE 10

The `useEffect` Hook lets you perform side effects in function components.

10.1 What is `useEffect()`?

- It replaces lifecycle methods (`componentDidMount`, etc.).
- Runs after the component renders.
- Used for data fetching, subscriptions, timers, DOM ops, etc.

```
useEffect(() => {
  // side effect here
  console.log("Component mounted / updated");

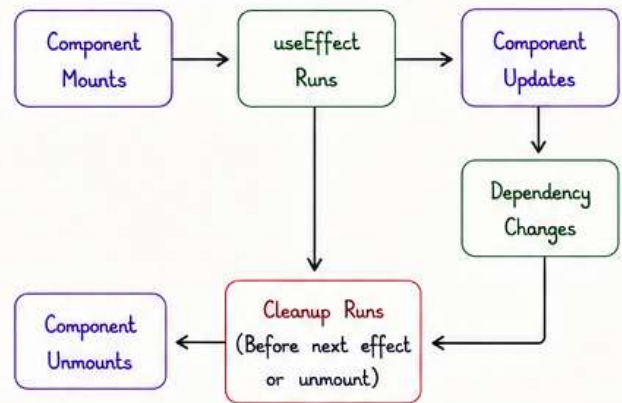
  return () => {
    // cleanup (optional)
    console.log("Cleanup before unmount / re-run");
  };
}, [dependency]);
```

← Effect

← Cleanup function

← Dependency array

10.2 `useEffect()` Lifecycle



★ Cleanup runs before the effect re-runs and when the component unmounts.

10.3 `useEffect()` Examples

1. Run on Mount (Once)

```
useEffect(() => {
  console.log("Mounted");
}, []); // Empty array -> run once
```

2. Run on Every Update (No dependency)

```
useEffect(() => {
  console.log("Runs on every render");
}); // No dependency -> runs every time
```

3. Run on Dependency Change

```
useEffect(() => {
  console.log("Runs when count changes");
}, [count]);
```

4. Cleanup Example

```
useEffect(() => {
  const id = setInterval(() => {
    console.log("Timer running");
  }, 1000);

  return () => clearInterval(id); // cleanup
}, []);
```

10.4 `useEffect()` Use Cases



- Fetching Data
- ```
useEffect(() => { fetchData(); }, []);
```



- Subscriptions (WebSocket, Events)
- ```
useEffect(() => {
  socket.connect();
  return () => socket.disconnect();
}, []);
```



- Timers & Intervals
- ```
useEffect(() => {
 const id = setInterval(() => {...}, 1000);
 return () => clearInterval(id);
}, []);
```



- DOM Manipulation
- ```
useEffect(() => {
  document.title = "React App";
}, []);
```

10.5 `useEffect()` Do's & Don'ts

Do's ✓	Don'ts ✗
✓ Use effect for side effects only.	✗ Don't forget dependency array.
✓ Keep dependency array accurate.	✗ Don't put async function directly in <code>useEffect</code> .
✓ Clean up subscriptions, timers, listeners.	✗ Don't cause infinite loops (by updating deps inside effect).
✓ Use <code>useCallback</code> / <code>useMemo</code> to optimize deps.	✗ Don't ignore cleanup when needed.
✓ Keep effect logic small and focused.	✗ Don't overuse <code>useEffect</code> for simple logic.

10.6 Lifecycle Methods vs `useEffect`

Class Component Method	When it Runs	<code>useEffect</code> Equivalent
<code>componentDidMount()</code>	After first render	<code>useEffect(() => {...}, [])</code>
<code>componentDidUpdate()</code>	After updates	<code>useEffect(() => {...}, [deps])</code>
<code>componentWillUnmount()</code>	Before unmount	Cleanup function in <code>useEffect</code>
<code>componentDidMount</code> + <code>componentDidUpdate</code>	On mount + updates	<code>useEffect</code> without dependency (⚠ not recommended)

10.7 Effect Execution Summary

Dependency Array	When it Runs	Cleanup Runs
<code>[]</code> (empty)	Once after first render (mount)	On unmount
<code>[a]</code>	On mount and when 'a' changes	Before next run & on unmount
No array	After every render	Before next run & on unmount

Note:

In React 18 (Strict Mode in development), `useEffect` runs twice on mount to help detect issues. (Only in development) 😊

COMMON MISTAKES

- ✗ Missing dependency array → effect runs every render.
- ✗ Incorrect dependencies → stale or unexpected values.
- ✗ Forgetting cleanup → memory leaks, duplicate listeners.
- ✗ Using effect for state sync → use derived state or memo.
- ✗ Triggering state update inside effect → infinite loop.



Tip: Think of `useEffect` as: "Do something after React updates the UI."
Use it wisely - it's powerful but easy to misuse!

IG: abhi_techhub





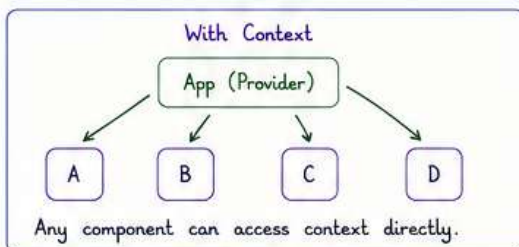
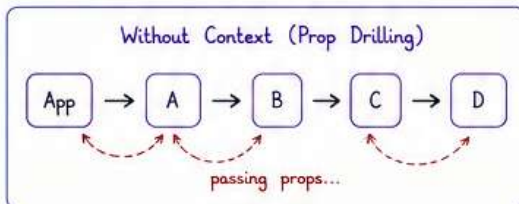
11. CONTEXT API

PAGE 11

Context API helps share data globally without prop drilling.

11.1 Why Context API?

- Props drilling is painful.
- Context provides data to any component, no matter how deep.
- Ideal for theme, auth, language, user info, etc.



11.2 Creating Context

```
import { createContext } from 'react';
// default value (optional)
export const UserContext = createContext(null);
```

← Create the Context

11.3 Providing Context (Provider)

```
import { useState } from 'react';
import { UserContext } from './UserContext';

export function UserProvider({ children }) {
  const [user, setUser] = useState({ name: 'Abhi', role: 'Dev' });
  return (
    <UserContext.Provider value={{ user, setUser }}>
      {children}
    </UserContext.Provider>
  );
}
```

← Wrap your app (or part of it) with Provider.

11.4 Consuming Context

1. Using useContext Hook

```
import { useContext } from 'react';
import { UserContext } from './UserContext';

export function Profile() {
  const { user, setUser } = useContext(UserContext);
  return (
    <div>
      <h3>Hello, {user.name}</h3>
      <p>Role: {user.role}</p>
      <button onClick={() => setUser({ ...user, role: 'Admin' })}>
        Make Admin
      </button>
    </div>
  );
}
```

← Access context values anywhere!

2. Using Context.Consumer (Old Way)

```
<UserContext.Consumer>
  {(value) => (
    <div>Hello, {value.user.name}</div>
  )}
</UserContext.Consumer>
```

← Works, but useContext is recommended.

11.5 Updating Context Values

- Update state in Provider.
- All consumers re-render automatically.

```
function changeName(newName) {
  setUser((prev) => ({ ...prev, name: newName }));
}
```

★ Only the components using the changed value will re-render, not the entire app.

11.6 Example Flow



★ Think of Context as a global state container for your component tree.

11.7 Common Use Cases

Use Case	Example
	Theme
	Light / Dark mode across the app
	Logged in user info
	Current language (i18n)
	User preferences (font, layout, etc.)
	Cart / Checkout
	Shopping cart data
	Socket instance or status

11.8 Best Practices

- ✓ Use Context for global, app-wide state.
- ✓ Keep context values stable (useMemo) to avoid unnecessary re-renders.
- ✓ Split context by domain (AuthContext, ThemeContext, etc.).
- ✓ Don't put everything in one context.
- ✓ Use custom hooks for cleaner access. (e.g., useAuth, useTheme)



DON'TS

- ✗ Don't use Context for frequently changing local state.
- ✗ Don't pass functions that change on every render (memoize).
- ✗ Don't overuse Context for small, isolated components.
- ✗ Don't ignore performance - large contexts can cause extra renders.



Tip: Context is powerful, but use it wisely. For complex or high-performance global state, explore Redux / Zustand / Recoil.

IG: abhi_techhub





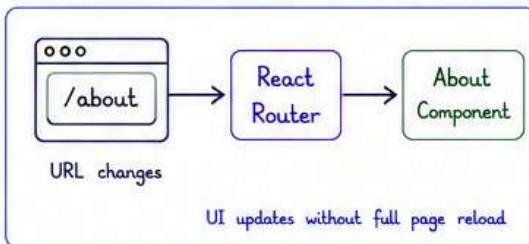
12. REACT ROUTER

PAGE 12

React Router helps build Single Page Applications (SPA) with navigation and routing.

12.1 Why React Router?

- Enables navigation without reloading the page.
- Maps URLs to components.
- Supports dynamic routes, nested routes, and route protection.



12.4 Link vs useNavigate

1. Link (for UI navigation)

```
import { Link } from 'react-router-dom';
<nav>
  <Link to="/">Home</Link> |
  <Link to="/about">About</Link>
</nav>
```

← Renders an
anchor tag
<a>

2. useNavigate (Programmatic navigation)

```
import { useNavigate } from 'react-router-dom';

function GoToAbout() {
  const navigate = useNavigate();
  return (
    <button onClick={() => navigate('/about')}>
      Go to About
    </button>
  );
}
```

← Navigate
inside
functions

12.6 Nested Routes

```
<Route path="/dashboard" element={DashboardLayout} />
<Route index element={DashboardHome} />
<Route path="/profile" element={Profile} />
<Route path="/settings" element={Settings} />
</Route>
```

Child routes
render inside
parent layout
using <Outlet />

DashboardLayout.jsx

```
import { Outlet } from 'react-router-dom';

function DashboardLayout() {
  return (
    <div>
      <h2>Dashboard</h2>
      <nav> ... </nav>
      <Outlet /> { /* Child routes render here */ }
    </div>
  );
}
```

12.2 Installation

```
# Using npm
npm install react-router-dom

# Using yarn
yarn add react-router-dom
```

v6+ is
recommended

12.3 Basic Setup

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={Home} />
        <Route path="/about" element={About} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

← Wrap your
app with
BrowserRouter

← Define all
routes inside
<Routes>

← Use element
prop to render
components

12.5 Dynamic Routes

```
// Define route with parameter
<Route path="/user/:id" element={User} />

// Access parameter in component
import { useParams } from 'react-router-dom';

function User() {
  const { id } = useParams();
  return <div>User ID: {id}</div>;
}
```

← :id is a
dynamic
parameter

← Access it
using
useParams()

12.7 404 Page (Not Found)

```
<Routes>
  <Route path="/" element={Home} />
  <Route path="/about" element={About} />
  <Route path="*" element={NotFound} />
</Routes>
```

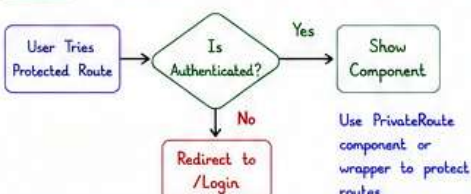
← * matches
any unknown
route

NotFound.jsx

```
function NotFound() {
  return (
    <div style={{ textAlign: 'center', marginTop: '2rem' }}>
      <h2>404 - Page Not Found</h2>
      <p>The page you're looking for doesn't exist.</p>
      <Link to="/">Go back Home</Link>
    </div>
  );
}
```



12.8 Route.Guards (Protected Routes)



12.9 Best Practices

- ✓ Keep routes in a separate file (routes.js).
- ✓ Use lazy loading for route components.
- ✓ Use meaningful and clean URLs.
- ✓ Handle 404 pages and errors gracefully.
- ✓ Protect sensitive routes (auth, admin).
- ✓ Keep navigation logic consistent.



COMMON MISTAKES

- ✗ Forgetting to wrap app with BrowserRouter.
- ✗ Using <a> instead of <Link>.
- ✗ Not handling 404 routes.
- ✗ Hardcoding URLs in multiple places.
- ✗ Forgetting to add exact/children routes.



Tip: Router is the backbone of SPA.
Design your routes well = Better UX & Scalable app.

IG: abhi_techhub

